```
    redirect_to article_path(@article)
  end

  private
    def comment_params
      params.require(:comment).permit(:commenter, :body)
    end
end
```

The `destroy` action will find the article we are looking at, locate the comment within the `@article.comments` collection, and then remove it from the database and send us back to the show action for the article.

8.1 Deleting Associated Objects

If you delete an article, its associated comments will also need to be deleted, otherwise they would simply occupy space in the database. Rails allows you to use the `dependent` option of an association to achieve this. Modify the Article model, `app/models/article.rb`, as follows:

```
class Article < ActiveRecord::Base
  has_many :comments, dependent: :destroy
  validates :title, presence: true,
                    length: { minimum: 5 }
end
```

9 Security

9.1 Basic Authentication

If you were to publish your blog online, anyone would be able to add, edit and delete articles or delete comments.

Rails provides a very simple HTTP authentication system that will work nicely in this situation.

In the `ArticlesController` we need to have a way to block access to the various actions if the person is not authenticated. Here we can use the Rails `http_basic_authenticate_with` method, which allows access to the requested action if that method allows it.
To use the authentication system, we specify it at the top of our `ArticlesController` in `app/controllers/articles_controller.rb`. In our case, we want the user to be authenticated on every action except `index` and `show`, so we write that:

```
class ArticlesController < ApplicationController

  http_basic_authenticate_with name: "dhh", password: "secret",
except: [:index, :show]

  def index
    @articles = Article.all
  end

  # snipped for brevity
```

We also want to allow only authenticated users to delete comments, so in the `CommentsController`(`app/controllers/comments_controller.rb`) we write:

```
class CommentsController < ApplicationController
```